

RNG com não repetição (3 Bits)

Relatório de síntese

Projeto do Bloco Digital

Matheus Grossi

Data: March 15, 2026
Rev.: 9.0

Contents

1	Síntese baseada em FPGA	2
2	Fluxo de Funcionamento e Timing	4
3	Síntese baseada em ASIC Flow no PDK Open-SKY130	5
3.1	Definições da estrutura para a síntese	5
3.2	Restrições de Design (SDC - Synopsys Design Constraints)	6
4	Resultados da Síntese Lógica e Física	6
4.1	Diagrama Lógico	6
4.2	Consumo de Potência	7
5	Soc-View	8
6	Simulação	9

Siglas e Acrónimos

RNG	Random Number Generator (Gerador de Números Aleatórios)
PRNG	Pseudo-Random Number Generator
ASIC	Application Specific Integrated Circuit
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine (Máquina de Estados Finita)
HDL	Hardware Description Language
RTL	Register Transfer Level
SDC	Synopsys Design Constraints

1 Síntese baseada em FPGA

Este design é implementado utilizando HDL padrão de indústria e estruturado para facilitar a integração e síntese.

- **Nome do bloco:** RNG Não Repetitivo (3 bits)
- **Linguagem de Descrição:** Verilog
- **Tecnologias Alvo:** FPGA e ASIC
- **Características Chave:** Interface de Handshake, 4 Seeds, Histórico de não-repetição.

Abaixo estão as vistas explodidas do design após síntese em FPGA:

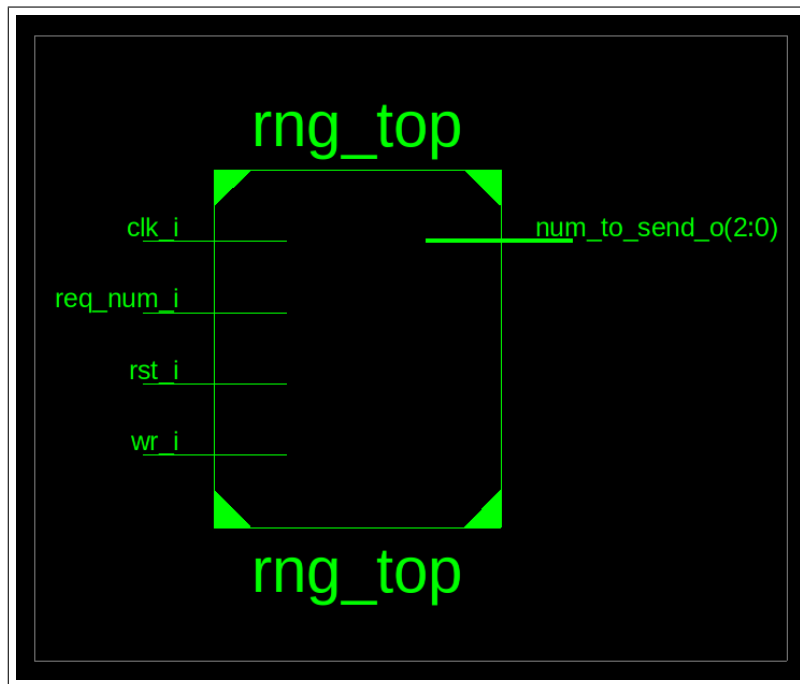


Figure 1: Visão de Topo do Circuito (Síntese)

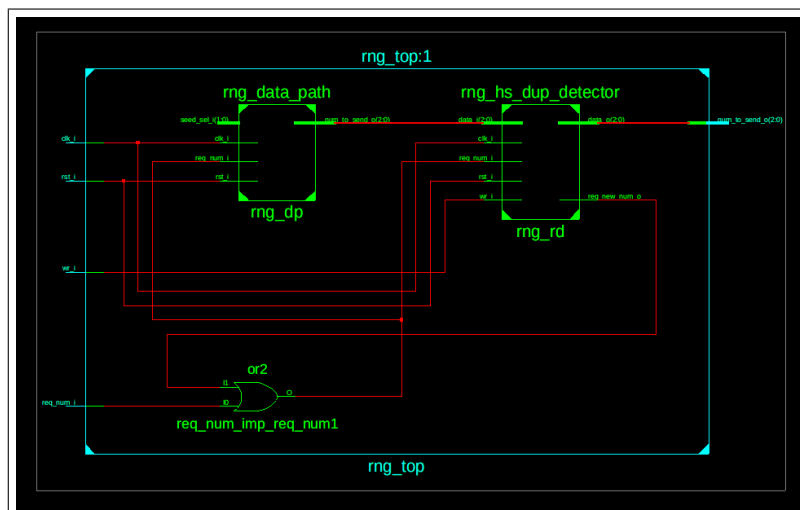


Figure 2: Vista Explodida 1

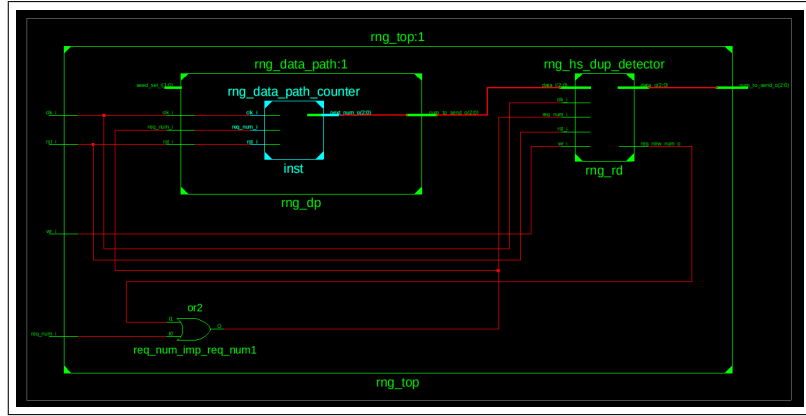


Figure 3: Vista Explodida 2

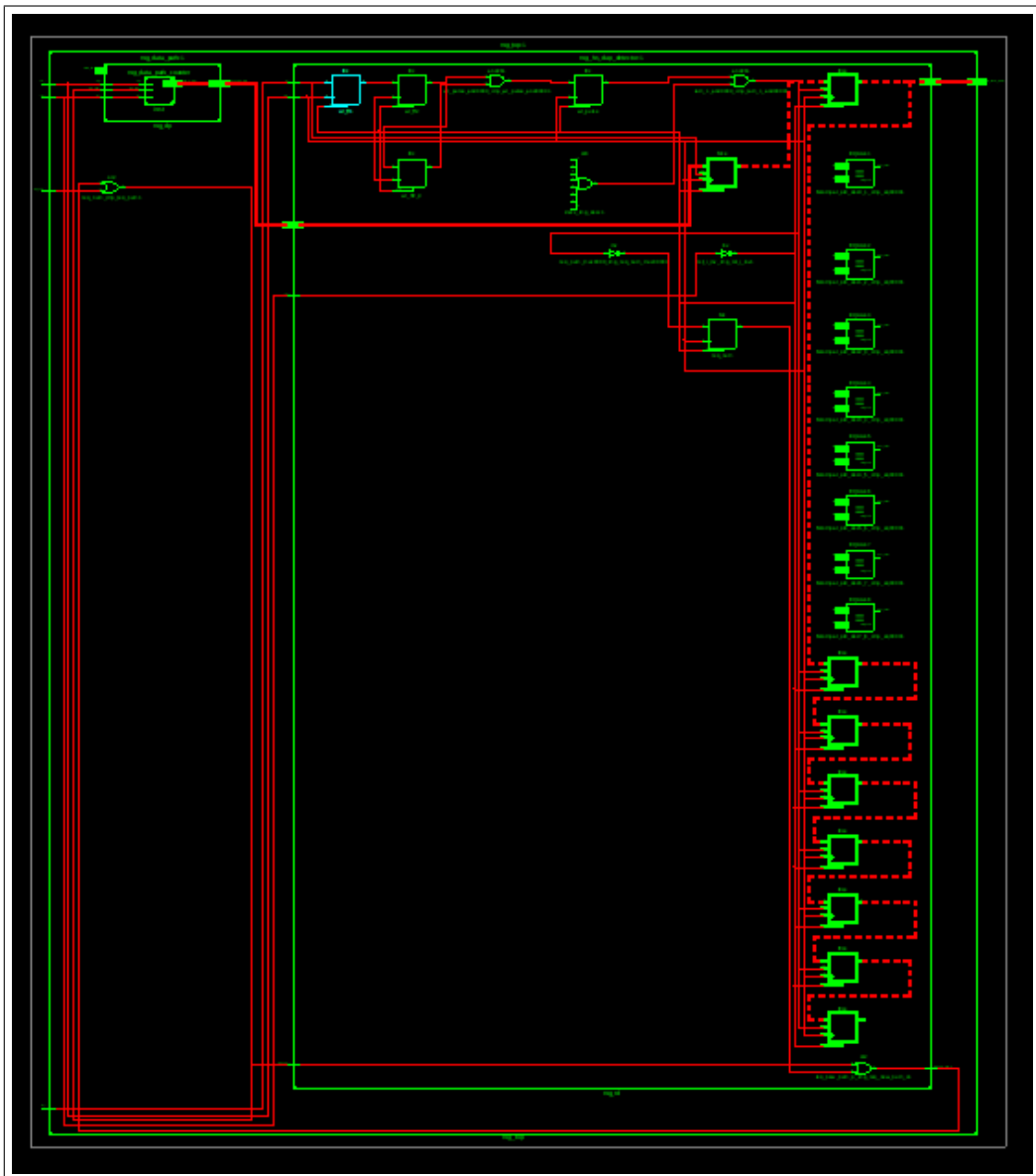


Figure 4: Vista Explodida 3

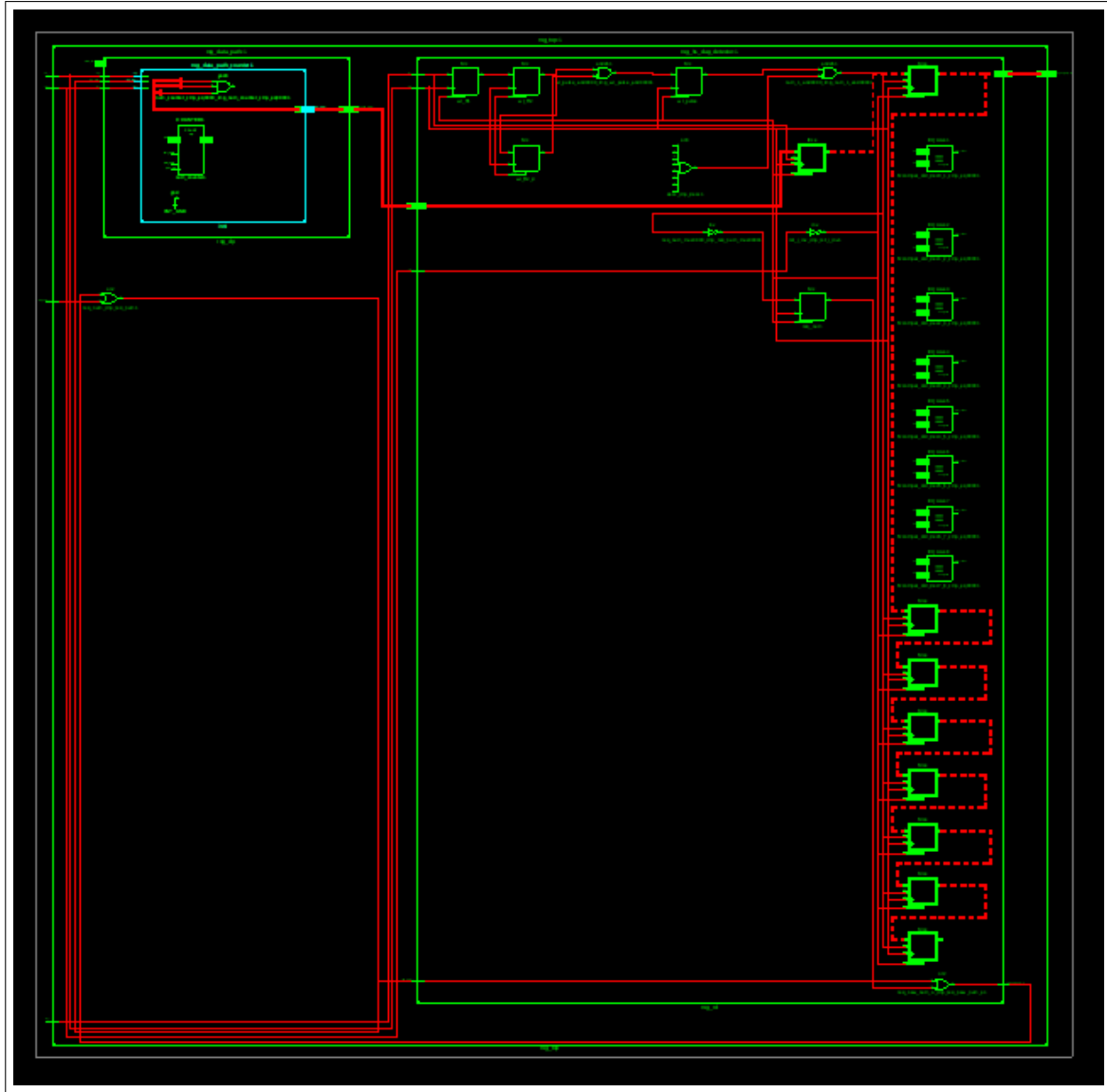


Figure 5: Vista Explodida 4

2 Fluxo de Funcionamento e Timing

O bloco opera através de um fluxo coordenado para garantir a validade dos dados:

1. **Reset:** O sistema é resetado (`rst_i`), limpando a memória do detetor de duplicatas.
2. **Seleção de Seed:** O módulo `rng_selector` utiliza um contador *free-running* para definir qual das 4 tabelas será usada.
3. **Geração e Validação:** O `rng_data_path` gera um candidato. O `rng_hs_dup_detector` verifica duplicatas. Se existir, descarta e busca o próximo. Se for único, disponibiliza na saída.
4. **Confirmação (Handshake):** O sinal `wr_i` confirma a leitura e salva o valor no histórico.

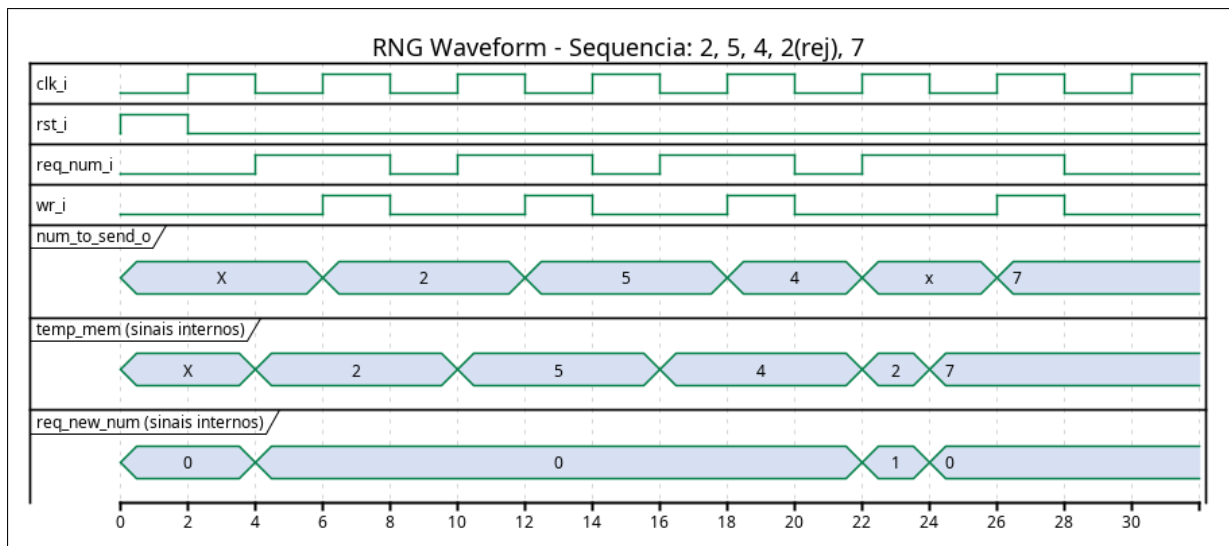


Figure 6: Diagrama de Temporização (Timing)

3 Síntese baseada em ASIC Flow no PDK Open-SKY130

Além da implementação em FPGA, o projeto foi submetido ao fluxo de design digital para ASIC utilizando o ambiente *Librelane*. O mapeamento tecnológico direcionou o circuito para a biblioteca de *standard cells* de código aberto *sky130_fd_sc_hd* (SkyWater 130nm High Density).

3.1 Definições da estrutura para a síntese

Para executar esse processo, siga os passos abaixo:

- Crie uma pasta com o nome do projeto dentro do diretório `openframe_user/openlane`.
- Internamente a esta pasta, crie um subdiretório chamado `src`.
- Na raiz da pasta do projeto, crie um arquivo nomeado `config.json`. Este script irá armazenar os dados primários de roteamento e síntese prévios ao SDC.

```
matheus@matheus-Vivobook-Go-E1504FA-E1504FA:~/openframe_user_project/openlane/rng$ ls -la
total 20
drwxr-xr-x  4 matheus matheus 4096 mar 12 20:01 .
drwxrwxr-x 11 matheus docker  4096 fev 17 10:42 ..
-rw-rw-r--  1 matheus matheus  439 mar  9 20:42 config.json
drwxr-xr-x  8 matheus matheus 4096 mar 12 20:08 runs
drwxr-xr-x  3 matheus matheus 4096 mar 12 23:00 src
```

Figure 7: Organização da pasta de projeto

- Dentro da pasta `src` insira os arquivos `.v` e `.sv` de seu projeto.
- Em meu caso deixei o `config.json` definido conforme exemplo na pagina oficial do sintetizador.

```

{
  "DESIGN_NAME": "rng_top",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk_i",
  "CLOCK_PERIOD": 3125,
  "RSZ_DONT_TOUCH": ["*_DONOTTOUCH"],
  "FP_SIZING": "absolute",
  "DIE_AREA": "0 0 70 85",
  "FP_CORE_UTIL": "50",
  "FP_ASPECT_RATIO": 1.0,
  "FP_CORE_MARGIN": 2,
  "PL_TARGET_DENSITY": 0.6,
  "FP_PDN_MULTILAYER": true,
  "SYNTH_ABC_LEGACY_REFACTOR": false,
  "SYNTH_ABC_LEGACY_REWRITE": 0
}

```

Figure 8: Configuração do script

3.2 Restrições de Design (SDC - Synopsys Design Constraints)

Devido ao fato de este projeto se tratar de uma implementação simples e que não passará por um tape-out foi feito o uso de um SDC genérico, e portanto as constraints usadas foram as padrões, o que implica que as otimizações e temporizações não foram as melhores para esta aplicação

4 Resultados da Síntese Lógica e Física

4.1 Diagrama Lógico

A estrutura do módulo `rng_top` consolida quatro sub-blocos principais: caminho de dados, seletor de semente, detetor de duplicatas e contador cíclico. A tradução lógica deste caminho de dados para o nível de portas resultou num uso intensivo de multiplexadores para o roteamento e portas exclusivas para validação e contagem. A arquitetura foi sintetizada em 122 células lógicas no total como pode ser observado na imagem abaixo.

```

=== rng_top ===

Number of wires:          106
Number of wire bits:      126
Number of public wires:   21
Number of public wire bits: 41
Number of ports:          5
Number of port bits:      7
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          122
  $_ANDNOT_                19
  $_AND_                    1
  $_MUX_                    30
  $_NAND_                   1
  $_ORNOT_                  1
  $_OR_                     9
  $_XNOR_                   8
  $_XOR_                    18
  sky130_fd_sc_hd__dfirtp_2 35

Chip area for module '\rng_top': 919.632000
  of which used for sequential elements: 919.632000 (100.00%)

```

Figure 9: Composição do SoC

4.2 Consumo de Potência

Os relatórios estáticos de síntese atuais (`stat.rpt` e `stat.json`) documentam detalhadamente a área, os tipos de células e as interconexões lógicas. Para avaliar com precisão o consumo de potência estática (*leakage*) e dinâmica, o fluxo requer uma extração pós-roteamento. Esta análise baseia-se na atividade de comutação (*switching activity*), sendo necessário fornecer os ficheiros da simulação (`.vcd`) gerados em `waveforms` a uma ferramenta de *Power Analysis* nativa do fluxo Libreane.


```
=====
report_power
=====
===== nom_tt_025C_1v80 Corner =====
```

Group	Internal Power	Switching Power	Leakage Power	Total Power (Watts)	
Sequential	6.595862e-07	1.464358e-07	4.117267e-10	8.064338e-07	33.7%
Combinational	6.884237e-07	4.226346e-07	5.090588e-10	1.111567e-06	46.5%
Clock	3.096065e-07	1.619649e-07	2.217573e-10	4.717931e-07	19.7%
Macro	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.0%
Pad	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.0%
Total	1.657616e-06	7.310353e-07	1.142543e-09	2.389794e-06	100.0%
	69.4%	30.6%	0.0%		

Figure 10: Métricas de Consumo de Potência

5 Soc-View

Este foi o resultado final de nossa síntese após todos os processos:

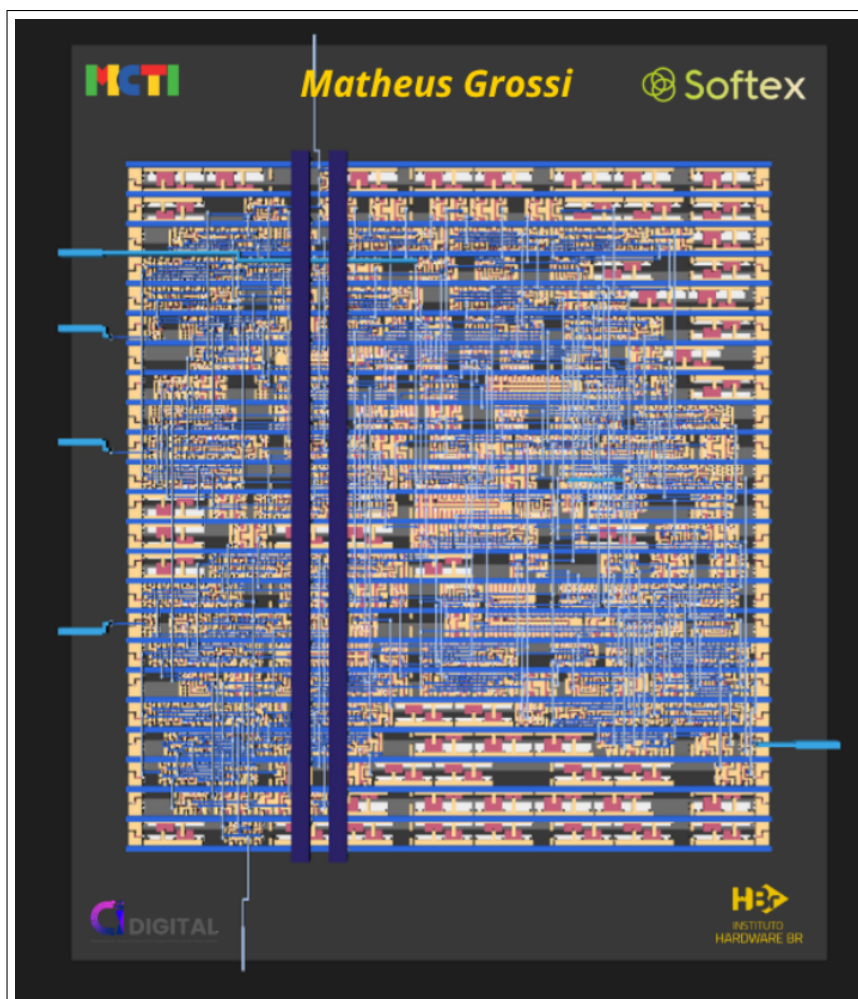


Figure 11: Waveform da Simulação

6 Simulação

Os ambientes de teste estão localizados em `testbenchs` e utilizam scripts de automação (Makefiles/Bash) para geração de ondas. O waveform abaixo demonstra o comportamento real obtido em simulação com a bancada rodando em nossa netlist pós-síntese.

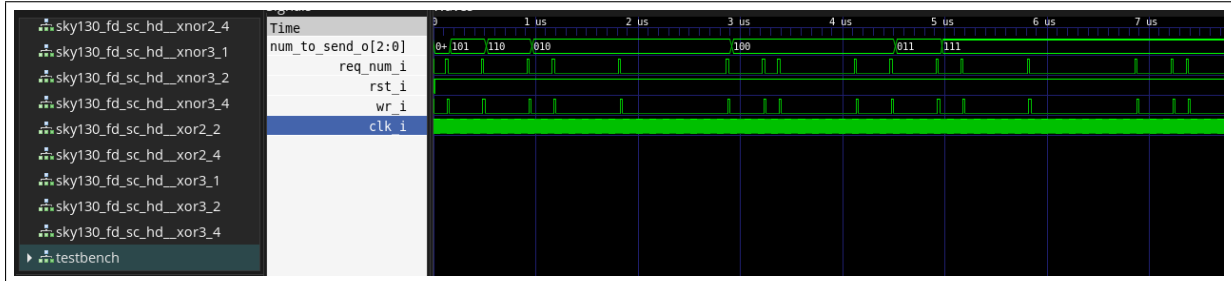


Figure 12: Waveform da Simulação

Metodologia: Design Síncrono com validação via Handshake e Testbenchs Automatizados.